Lập trình mạng · Chương 2

Đa luồng: thread và race condition

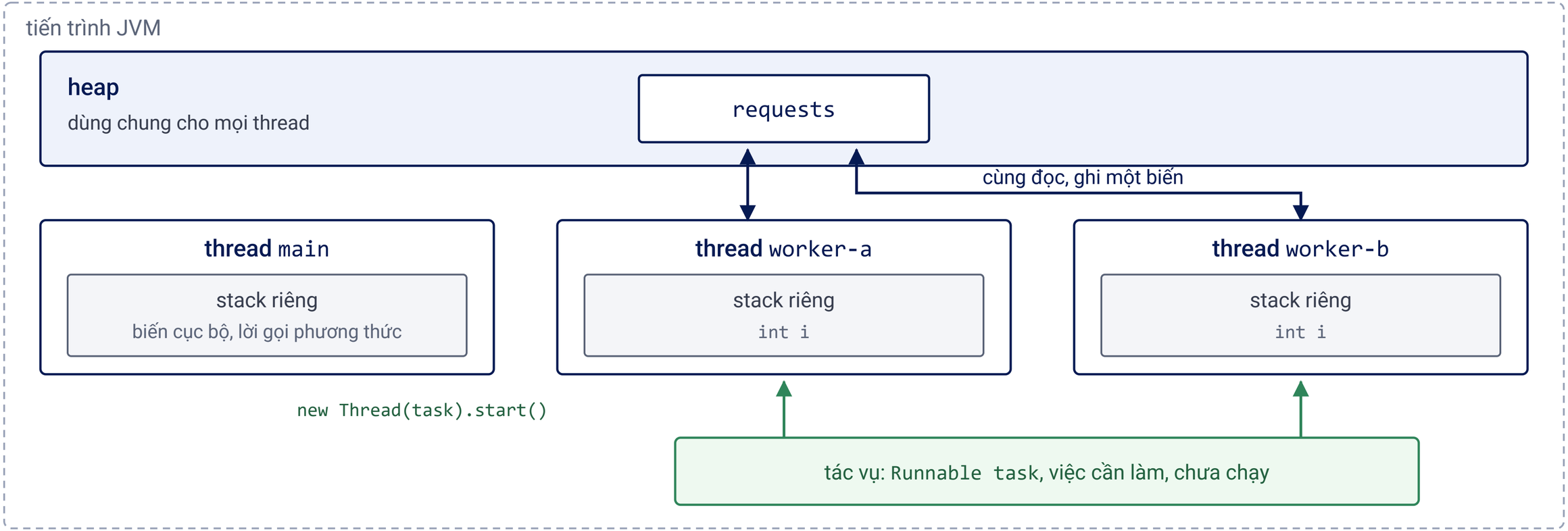
TS. Đặng Ngọc Hùng

Xử lý mỗi kết nối trên một thread riêng, nhưng các thread dùng chung dữ liệu

1. Server tuần tự: một client không gửi gì làm client sau chờ **khoảng 4 giây**
2. Hướng giải quyết: xử lý mỗi kết nối trên một **thread** riêng, các kết nối chạy **song song**
3. Các thread **dùng chung dữ liệu**, ví dụ bộ đếm số yêu cầu đã phục vụ

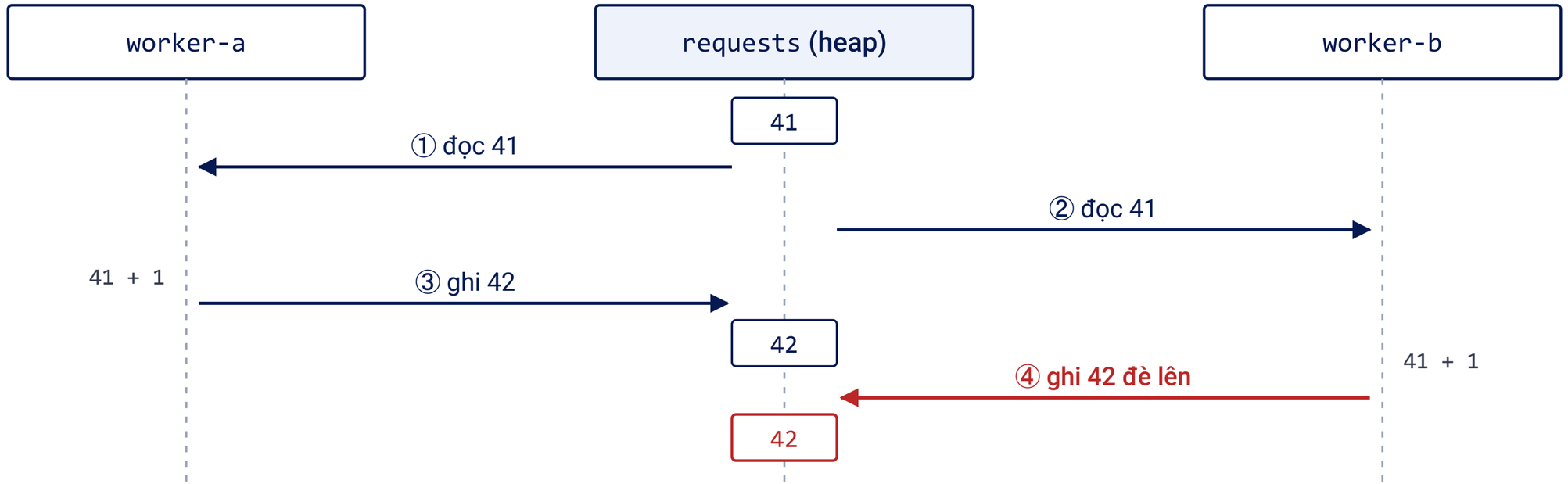
> **Câu hỏi của phần:** tạo, chạy và dừng một thread thế nào, và vì sao nhiều thread cùng cập nhật một biến có thể cho kết quả sai?

Các thread trong một tiến trình có stack riêng và dùng chung heap



Thread: stack riêng, heap dùng chung. **Tác vụ** (`Runnable`) mô tả việc cần làm; `start()` cho tác vụ chạy trên một thread mới.

Tăng bộ đếm gồm đọc, sửa, ghi: hai thread xen kẽ thì mất một lần tăng



Tăng biến dùng chung là **đọc – sửa – ghi**; hai thread xen kẽ giữa các bước thì mất cập nhật: **race condition**.

Hai thread tăng bộ đếm, kết quả thiếu và khác nhau mỗi lần chạy

Java

```
Runnable task = () -> {  // tác vụ: mô tả việc cần làm, chưa chạy
    for (int i = 0; i < 1_000_000; i++) {
        requests++;  // đọc – sửa – ghi, không đồng bộ hóa
    }
};
Thread a = new Thread(task, "worker-a");
Thread b = new Thread(task, "worker-b");
a.start();  // tạo thread mới, chạy task song song với main
b.start();
a.join();  // main chờ hai thread kết thúc
b.join();
```

Kết quả chạy

```
run 1: requests = 1906790 (expected 2000000)
run 2: requests = 1814219 (expected 2000000)
run 3: requests = 1905748 (expected 2000000)
```

interrupt() yêu cầu thread dừng, thread tự kết thúc

Java

```
static void waitForWork() {
    try {
        while (!Thread.currentThread().isInterrupted()) {  // kiểm tra trạng thái ngắt
            System.out.println("worker: waiting for work");
            Thread.sleep(1_000);  // thao tác chờ: ném InterruptedException khi bị ngắt
        }
    } catch (InterruptedException e) {  // yêu cầu dừng: dọn dẹp rồi kết thúc
        System.out.println("worker: interrupted, cleaning up");
    }
}
```

Kết quả chạy

```
worker: waiting for work
worker: waiting for work
worker: waiting for work
worker: interrupted, cleaning up
main: worker alive = false
```

- `interrupt()` đánh thức thread đang chờ trong `sleep`, `wait`, `join`; thread đang tính toán phải **tự kiểm tra** `isInterrupted()`

Gọi run() không tạo thread mới, và Thread.stop() không còn dùng được

```java
Runnable task = () -> System.out.println("task runs on " + Thread.currentThread().getName());
Thread worker = new Thread(task, "worker");
worker.run();    // gọi nhầm: chạy tác vụ trên thread hiện tại (main)
worker.start();  // đúng: tạo thread mới tên worker
worker.join();
worker.stop();  // không còn dùng được trong Java 21
```

Kết quả chạy

```
task runs on main
task runs on worker
Thread.stop(): UnsupportedOperationException
```

- `run()` chỉ là lời gọi phương thức: tác vụ chạy **tuần tự** trên thread đang gọi
- `Thread.stop()` ném `UnsupportedOperationException` từ Java 20: dừng thread bằng **interrupt**

Câu hỏi 1

Hai thread cùng thực hiện requests++ một triệu lần trên một biến dùng chung, không đồng bộ hóa. Vì sao kết quả có thể nhỏ hơn hai triệu?

A. Vì một thread chạy xong trước khi thread kia bắt đầu.

B. Vì requests++ gồm đọc, cộng, ghi; hai thread đọc cùng một giá trị rồi ghi đè lên nhau.

C. Vì kiểu int bị tràn số khi vượt một triệu.

D. Vì JVM bỏ qua một số lần lặp để chạy nhanh hơn.

Câu hỏi 1

Hai thread cùng thực hiện requests++ một triệu lần trên một biến dùng chung, không đồng bộ hóa. Vì sao kết quả có thể nhỏ hơn hai triệu?

A. Vì một thread chạy xong trước khi thread kia bắt đầu.

B. Vì requests++ gồm đọc, cộng, ghi; hai thread đọc cùng một giá trị rồi ghi đè lên nhau.

C. Vì kiểu int bị tràn số khi vượt một triệu.

D. Vì JVM bỏ qua một số lần lặp để chạy nhanh hơn.

requests++ là đọc – sửa – ghi. Khi hai thread xen kẽ giữa bước đọc và bước ghi, một lần tăng bị ghi đè. Nếu hai thread chạy lần lượt (phương án A) thì kết quả lại đúng; int chứa được hai triệu.

Câu hỏi 2

Một chương trình tạo new Thread(task) rồi gọi `run()` thay cho `start()`. Điều gì xảy ra?

A. Tác vụ chạy trên một thread mới, giống như `start()`.

B. Chương trình báo lỗi biên dịch.

C. Tác vụ không chạy.

D. Tác vụ chạy trên thread hiện tại; không có thread mới nào được tạo.

Câu hỏi 2

Một chương trình tạo new Thread(task) rồi gọi `run()` thay cho `start()`. Điều gì xảy ra?

A. Tác vụ chạy trên một thread mới, giống như `start()`.

B. Chương trình báo lỗi biên dịch.

C. Tác vụ không chạy.

D. Tác vụ chạy trên thread hiện tại; không có thread mới nào được tạo.

`run()` là một phương thức bình thường: gọi nó thì tác vụ chạy ngay trên thread đang gọi. Chỉ `start()` mới tạo thread mới.

Tổng kết

1. **Thread**: stack riêng, heap dùng chung; **tác vụ** `Runnable` chạy trên thread mới khi gọi `start()`

2. Dừng thread bằng **interrupt**: thread tự kiểm tra trạng thái ngắt, hoặc nhận `InterruptedException` khi đang chờ

3. **Race condition**: nhiều thread đọc – sửa – ghi dữ liệu dùng chung, xen kẽ thì mất cập nhật

Từ khóa cần nhớ thread · tác vụ · Runnable · interrupt · race condition

Mở rộng

Chủ đề nâng cao

- AtomicInteger và thao tác compare-and-set
- Công cụ kiểm thử đồng thời jcstress

Bài tập mở rộng

Viết chương trình có ba thread cùng thêm phần tử vào một ArrayList; quan sát kết quả sai (thiếu phần tử, ngoại lệ) và giải thích.

Tiếp theo

Đa luồng: synchronized, happens-before và wait/notify